



Facultad de Ingeniería
Departamento de Ingeniería en Computación e
Informática

Divide y Vencerás

Ivan Mansilla (ivnmansi@umag.cl)

Diego Peralta (dperalta@umag.cl)

Catalina Viñas (caastudi@umag.cl)

Ingeniería en Computación e Informática

Docente: MSc. Jacqueline Aldridge

Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de una expansión a un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV, utilizando algoritmos de ordenamiento y búsqueda. A este se implementaron algoritmos que utilizan el paradigma de divide y vencerás, tal como Merge-Sort, QuickSort, etc. Con esto hecho, se analizó experimentalmente el rendimiento de cada algoritmo, comparando sus tiempos de ejecución y creando gráficos para visualizar tanto sus diferencias como sus complejidades temporales.

ÍNDICE GENERAL

Índice de figuras	v
0.1. Introducción	1
1. Objetivos	2
1.1. Objetivo Principal	2
1.2. Objetivos Secundarios	2
2. Marco Teórico	3
2.1. Divide y Vencerás	3
2.2. Algoritmos de Ordenamiento	3
2.2.1. Merge Sort	3
2.2.2. Merge Sort Optimizado	3
2.2.3. Quick sort	4
2.2.3.1. Pivote último elemento	4
2.2.3.2. Pivote primer elemento	4
2.2.3.3. Pivote elemento aleatorio	4
2.2.3.4. Pivote mediana de tres	5
2.3. Algoritmos de Búsqueda y Selección	5
2.3.1. Búsqueda Binaria Recursiva	5
2.3.2. Búsqueda por Interpolación	5
2.3.3. Búsqueda exponencial	5
2.3.4. Búsqueda binaria por rango	6
2.3.5. Quick Select	6
3. Desarrollo	7
3.1. Diseño general del sistema	7
3.1.1. Deportistas y generación de datos	7
3.1.2. Funcionalidades	7
3.2. Algoritmos de Ordenamiento	8
3.2.1. Merge Sort	8
3.2.2. Merge Sort Optimizado	9
3.2.3. Quick sort	10
3.2.3.1. Partición de Lomuto	10

3.2.3.2.	Variantes de selección de pivote	10
3.2.3.3.	Variante de último elemento	10
3.2.3.4.	Variante de primer elemento	10
3.2.3.5.	Variante de elemento aleatorio	10
3.2.3.6.	Variante de mediana de tres	11
3.2.3.7.	Estructura recursiva	11
3.3.	Algoritmos de Búsqueda y Selección	11
3.3.1.	Búsqueda Binaria Recursiva	11
3.3.1.1.	Caso base	12
3.3.1.2.	División	12
3.3.1.3.	Conquista	12
3.3.2.	Búsqueda exponencial	13
3.3.2.1.	Fase de expansión exponencial	13
3.3.2.2.	Fase de búsqueda binaria recursiva	13
3.3.3.	Búsqueda por Interpolación	13
3.3.3.1.	Caso base	14
3.3.3.2.	División	14
3.3.3.3.	Conquista	14
3.3.4.	Búsqueda por Rango	15
3.3.4.1.	Estructura recursiva	15
3.3.4.2.	Tipo de dato retornado	16
3.3.5.	Quick Select	16
3.3.5.1.	Selección de pivote con mediana de tres	17
3.3.5.2.	Partición de Lomuto	17
3.3.5.3.	Estructura recursiva de divide y vencerás	17
3.3.5.4.	Complejidad	18
3.4.	Aplicación práctica	18
3.4.1.	Ranking de los mejores N deportistas	18

ÍNDICE DE FIGURAS

0.1. Introducción

Los algoritmos con la estrategia "Divide y Vencerás" permiten resolver problemas complejos dividiéndolos en subproblemas más pequeños independientes resolviéndolos de forma recursiva y combinando las soluciones para obtener un resultado final. Esta técnica permite mejorar la eficiencia de trabajo, ya que comúnmente se requiere menos código, lo que a su vez permite reducir la complejidad de lectura de este facilitando así la depuración.

Entre los ejemplos más conocidos de algoritmos que utilizan "Divide y Vencerás" se encuentran Merge Sort, Quick Sort y la búsqueda binaria, los cuales destacan por su eficiencia en comparación con métodos tradicionales. Gracias a estas características, esta técnica se ha convertido en una herramienta fundamental en el desarrollo de software y en la resolución eficiente de problemas computacionales complejos.

OBJETIVOS

1.1. Objetivo Principal

Expandir un sistema de ordenamiento y búsqueda de deportistas, agregando algoritmos que utilizan la estrategia de divide y vencerás, con el fin de mejorar la eficiencia del sistema.

1.2. Objetivos Secundarios

- Implementar correctamente los algoritmos de ordenamiento y sus versiones optimizadas según corresponda.
- Implementar correctamente los algoritmos de búsqueda.
- Implementar correctamente el algoritmo de selección (QuickSelect).
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados.
- Evaluar y comprender la complejidad teórica de los algoritmos de ordenamiento, selección y búsqueda.
- Documentar de manera adecuada el código desarrollado y los resultados obtenidos en el análisis experimental.

MARCO TEÓRICO

En este capítulo se describen los conceptos y algoritmos utilizados en el proyecto, centrados en la estrategia de diseño de algoritmos conocida como "Divide y Vencerás".

2.1. Divide y Vencerás

'Divide y Vencerás' es una técnica de diseño de algoritmos que se basa en dividir un problema en subproblemas más pequeños, resolver cada subproblema de manera independiente y luego combinar las soluciones para obtener la solución del problema original. Esta estrategia es especialmente útil para problemas que pueden ser descompuestos de manera recursiva, como los algoritmos de ordenamiento y búsqueda.

Dividir el problema en partes más manejables permite reducir la complejidad y mejorar la eficiencia de los algoritmos. Además, esta técnica facilita la implementación de algoritmos recursivos, lo que puede simplificar el código y hacerlo más legible.

2.2. Algoritmos de Ordenamiento

2.2.1. Merge Sort

Es un algoritmo basado en la técnica de dividir el problema en subproblemas más pequeños. El proceso consiste en dividir el arreglo original en dos mitades, ordenarlas por separado de manera recursiva y finalmente combinarlas mediante una operación de mezcla (merge) que asegura el orden de los elementos.

Este algoritmo tiene una complejidad temporal de $O(n \log n)$ en todos los casos (mejor, promedio y peor), lo que lo hace eficiente y predecible para grandes conjuntos de datos. Esto se debe a su estructura regular de división recursiva, que no depende del estado inicial de los datos. Además, es un algoritmo estable, aunque su implementación estándar utiliza espacio adicional para la mezcla.

2.2.2. Merge Sort Optimizado

El rendimiento del Merge Sort estándar puede mejorar mediante ajustes prácticos. Una optimización común consiste en establecer un umbral; cuando el tamaño del arreglo es pequeño, el algoritmo cambia a *Insertion Sort* para reducir la carga de la recursividad. También es

posible verificar si las dos mitades ya están ordenadas antes de intentar mezclarlas, evitando operaciones innecesarias.

La complejidad asintótica sigue siendo $O(n \log n)$ en todos los casos. Sin embargo, las optimizaciones prácticas (como verificar si las mitades ya están ordenadas) reducen las constantes multiplicativas y mejoran el rendimiento en la práctica, aunque no cambian la complejidad asintótica del algoritmo.

2.2.3. Quick sort

Al igual que el Merge Sort, el Quick Sort es un algoritmo de ordenamiento basado en la técnica de divide y vencerás. Este algoritmo funciona seleccionando un pivote y realizando una partición del arreglo en dos subarreglos: uno con los elementos menores que el pivote y otro con los elementos mayores. Posteriormente, el mismo procedimiento se aplica de forma recursiva a cada subarreglo hasta que todos los elementos quedan ordenados.

2.2.3.1. Pivote último elemento

En esta variante, el pivote se selecciona como el último elemento del arreglo. Aunque es fácil de implementar, esta estrategia puede llevar a un rendimiento deficiente (complejidad $O(n^2)$) en casos donde los datos ya están ordenados o casi ordenados, ya que el pivote no divide el arreglo de manera equilibrada.

2.2.3.2. Pivote primer elemento

En esta variante, el pivote se selecciona como el primer elemento del arreglo. Similar a la selección del último elemento, esta estrategia también puede resultar en un rendimiento deficiente (complejidad $O(n^2)$) para datos ordenados o casi ordenados, debido a la falta de equilibrio en la partición.

2.2.3.3. Pivote elemento aleatorio

En esta variante, el pivote se selecciona de manera aleatoria dentro del arreglo. Esta estrategia mejora el rendimiento esperado del Quick Sort, ya que reduce la probabilidad de seleccionar pivotes que conduzcan a particiones desequilibradas. La complejidad promedio esperada es $O(n \log n)$, el mejor caso también es $O(n \log n)$ y el peor caso sigue siendo $O(n^2)$, aunque ocurre con baja probabilidad.

2.2.3.4. Pivote mediana de tres

En esta variante, el pivote se selecciona como la mediana de tres elementos: el primer, el último y el elemento central del arreglo. Esta estrategia busca elegir un pivote más representativo para reducir la probabilidad de particiones desequilibradas, especialmente cuando los datos están ordenados o casi ordenados. La complejidad promedio esperada es $O(n \log n)$, el mejor caso también es $O(n \log n)$ y el peor caso sigue siendo $O(n^2)$, aunque con menor probabilidad que en las variantes de pivote fijo.

2.3. Algoritmos de Búsqueda y Selección

2.3.1. Búsqueda Binaria Recursiva

Es un algoritmo de búsqueda cuyo funcionamiento se basa en dividir repetidamente el arreglo en dos mitades, descartando en cada paso la mitad en la que el elemento buscado sea imposible de encontrar. Para funcionar necesita que los datos estén previamente ordenados.

La complejidad temporal de la búsqueda binaria es $O(\log n)$ en el mejor y promedio de los casos, y $O(\log n)$ en el peor caso, debido a la naturaleza de la división del espacio de búsqueda. La búsqueda binaria es eficiente para grandes conjuntos de datos ordenados, pero no es adecuada para datos no ordenados.

2.3.2. Búsqueda por Interpolación

Es una mejora sobre la búsqueda binaria para casos donde los datos están distribuidos de manera uniforme. En lugar de buscar siempre en el centro, el algoritmo calcula una posición estimada basándose en el valor buscado y los valores en los extremos del arreglo.

La complejidad temporal de la búsqueda por interpolación es $O(1)$ en el mejor caso, $O(\log \log n)$ en el caso promedio cuando los datos están distribuidos de manera uniforme, y $O(n)$ en el peor caso, especialmente cuando los datos no están distribuidos uniformemente. Este algoritmo puede ser significativamente más rápido que la búsqueda binaria para conjuntos de datos grandes y uniformemente distribuidos, pero su rendimiento puede degradarse considerablemente para datos no uniformes.

2.3.3. Búsqueda exponencial

Es un algoritmo de búsqueda que combina la eficiencia de la búsqueda binaria con la capacidad de manejar arreglos de tamaño desconocido. Comienza buscando en posiciones

exponenciales (1, 2, 4, 8, etc.) hasta encontrar un rango que contenga el elemento buscado. Luego, realiza una búsqueda binaria dentro de ese rango.

Su complejidad temporal es $O(1)$ en el mejor caso cuando el elemento buscado está en la primera posición, $O(\log i)$ en función de la posición i del elemento buscado, y $O(\log n)$ en el peor caso. La búsqueda exponencial es especialmente útil para arreglos de tamaño desconocido o muy grandes, donde la búsqueda binaria tradicional no sería aplicable.

2.3.4. Búsqueda binaria por rango

Es una variante de la búsqueda binaria que se utiliza para encontrar el rango de posiciones donde un valor específico aparece en un arreglo ordenado. En la implementación del proyecto, primero se localiza una coincidencia mediante búsqueda binaria y luego se expanden los límites hacia la izquierda y la derecha para abarcar todas las ocurrencias contiguas del mismo valor.

La complejidad temporal es $O(\log n + k)$, donde k es la cantidad de elementos que forman el rango encontrado. En el peor caso, cuando todas las posiciones coinciden con el valor buscado, la complejidad llega a $O(n)$. Para el caso de intervalos de puntaje, la implementación usa dos búsquedas binarias independientes para hallar los límites inferior y superior, por lo que cada extremo cuesta $O(\log n)$ y el total sigue siendo $O(\log n)$.

2.3.5. Quick Select

Este algoritmo se utiliza para encontrar el k -ésimo elemento más pequeño en una lista desordenada. Utiliza una lógica similar a Quick Sort (selección de un pivote), pero a diferencia de este, solo continúa la búsqueda en la partición que contiene el índice deseado, lo que reduce significativamente el tiempo de ejecución.

La complejidad temporal de Quick Select es $O(n)$ en el mejor caso, $O(n)$ en el caso promedio, y $O(n^2)$ en el peor caso. En la implementación del proyecto se utiliza la mediana de tres como criterio para elegir el pivote, lo que ayuda a reducir la probabilidad de alcanzar el peor caso y mejora el comportamiento práctico del algoritmo. Este algoritmo es especialmente útil para grandes conjuntos de datos donde se necesita encontrar un elemento específico sin ordenar completamente la lista.

DESARROLLO

3.1. Diseño general del sistema

El desarrollo se realizó sobre una versión anterior ya funcional del proyecto. Este consiste en un sistema de gestión de deportistas, que permite cargar datos desde un archivo CSV, ordenarlos según distintos criterios y mostrar resultados específicos como el ranking de los mejores deportistas. Este se basó en la idea de evaluar el rendimiento de diferentes algoritmos de ordenamiento y búsqueda secuenciales, los cuales se mantuvieron en la versión actual.

3.1.1. Deportistas y generación de datos

Los deportistas se representan mediante una estructura que contiene: *ID*, *nombre*, *puntaje* y *competencias*. La generación de datos se realiza mediante la opción `-g <numero>` del programa, que crea un archivo CSV con el número especificado de deportistas. Cada deportista tiene un ID único, un nombre generado aleatoriamente, y un puntaje y cantidad de competencias también aleatorios dentro de rangos predefinidos. Esto permite probar los algoritmos con conjuntos de datos de diferentes tamaños y características.

3.1.2. Funcionalidades

El programa implementa una interfaz de línea de comandos, que ofrece las siguientes funcionalidades principales:

- **Generación de datos:** Permite crear archivos CSV con un número específico de deportistas aleatorios.
- **Ordenamiento:** Dispone de un menú en el que el usuario puede ordenar los deportistas, seleccionando el algoritmo, el criterio de ordenamiento (ID, puntaje, competencias) y el orden (ascendente o descendente).
- **Búsqueda:** Permite buscar deportistas por ID, seleccionando el algoritmo de búsqueda. Si se selecciona la búsqueda binaria por rango, sin embargo este buscará por puntaje, mostrando todos los deportistas que tengan el puntaje ingresado.
- **Ranking:** Permite mostrar el ranking de los mejores N deportistas según puntaje, utilizando Quick Select para obtener los resultados de manera eficiente.

- **Búsqueda de k-ésimo elemento:** Permite obtener el deportista ubicado en la posición k según un criterio específico, también utilizando Quick Select.
- **Búsqueda por rango de puntajes:** Permite mostrar todos los deportistas cuyo puntaje se encuentre dentro de un intervalo definido por el usuario, utilizando búsqueda binaria por rango.
- **Benchmarks:** En el programa se incluyen 4 benchmarks de experimentación, los cuales miden el tiempo de ejecución de los algoritmos de ordenamiento, búsqueda y selección sobre conjuntos de datos de diferentes tamaños y características, guardando los resultados en archivos CSV para su posterior análisis.

3.2. Algoritmos de Ordenamiento

3.2.1. Merge Sort

La implementación de Merge Sort se realizó mediante una estrategia recursiva. El algoritmo divide sucesivamente el arreglo de deportistas en dos mitades hasta alcanzar el caso base, correspondiente a subarreglos de un único elemento, los cuales ya se consideran ordenados.

La función recursiva calcula el punto medio utilizando la expresión:

$$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$$

evitando posibles errores cuando los índices son muy grandes que podrían ocurrir con la expresión clásica:

$$(\text{left} + \text{right}) / 2$$

Una vez ordenadas ambas mitades, se utiliza una función auxiliar **merge** encargada de fusionarlas en un único segmento ordenado. Para ello, se reservan dinámicamente dos arreglos temporales (L y R) que almacenan copias de las sublistas izquierda y derecha respectivamente.

Durante la mezcla, los elementos son comparados mediante la función **compare_by_criteria**, permitiendo ordenar según distintos criterios del sistema (ID, puntaje o cantidad de competencias). Además, la función **should_precede** abstrae la lógica de comparación para soportar tanto orden ascendente como descendente.

El proceso de mezcla recorre simultáneamente ambos arreglos temporales e inserta en el arreglo original el elemento que debe preceder al otro según el criterio seleccionado. Cuando

una de las mitades se agota, los elementos restantes de la otra mitad son copiados directamente.

Debido al uso de memoria auxiliar para las sublistas temporales, el algoritmo requiere una complejidad espacial adicional de:

$$O(n)$$

mientras que su complejidad temporal se mantiene estable en:

$$O(n \log n)$$

tanto en el caso promedio como en el peor caso.

3.2.2. Merge Sort Optimizado

La versión optimizada de Merge Sort conserva la misma estructura recursiva y el mismo procedimiento de mezcla que la implementación estándar, pero incorpora una optimización híbrida basada en Insertion Sort para mejorar el rendimiento práctico sobre subarreglos pequeños.

Durante la recursión, antes de seguir dividiendo el arreglo, se evalúa el tamaño del subarreglo actual mediante la condición:

```
right - left + 1 <= MERGE_SORT_THRESHOLD
```

Si el tamaño es menor o igual al umbral definido, el algoritmo deja de subdividir y utiliza **Insertion Sort** sobre ese segmento.

Esta optimización se basa en que Insertion Sort posee un costo constante muy bajo y un excelente desempeño en arreglos pequeños o parcialmente ordenados, evitando el overhead asociado a múltiples llamadas recursivas y asignaciones dinámicas de memoria.

La implementación de **Insertion Sort** desplaza los elementos mayores hacia la derecha hasta encontrar la posición correcta del elemento actual (**key**). Al igual que en Merge Sort, las comparaciones se realizan mediante la función **should_precede**, manteniendo compatibilidad con múltiples criterios y órdenes de ordenamiento.

La operación de mezcla sigue utilizando arreglos auxiliares dinámicos para fusionar las mitades ya ordenadas. Además, la implementación incluye validaciones de memoria después de cada llamada a **malloc**. En caso de error de asignación, se liberan inmediatamente los recursos reservados mediante **free**, evitando fugas de memoria.

Aunque la complejidad asintótica permanece en:

$$O(n \log n)$$

la reducción de llamadas recursivas y operaciones de memoria sobre subarreglos pequeños produce una mejora significativa en el rendimiento real del algoritmo.

3.2.3. Quick sort

3.2.3.1. Partición de Lomuto

La partición es la operación central del algoritmo. Recibe el subarreglo delimitado por low y high, tomando siempre el último elemento como pivote. Mantiene un índice i que comienza antes del inicio del subarreglo y marca el límite de los elementos que ya se sabe que deben ir a la izquierda del pivote. El índice j recorre todos los elementos del subarreglo excepto el pivote. Cada vez que encuentra un elemento que pertenece a la izquierda, avanza i e intercambia ese elemento con deportistas[i]. Al terminar el recorrido, coloca el pivote en i + 1, que es su posición definitiva, y retorna ese índice.

3.2.3.2. Variantes de selección de pivote

Todas las variantes comparten exactamente el mismo código de partición y recursión. La única diferencia entre ellas es qué hacen antes de partir, y todas terminan dejando el pivote elegido en la última posición para que Lomuto pueda operar siempre de la misma manera.

3.2.3.3. Variante de último elemento

La variante de último elemento no realiza ninguna acción previa, ya que el pivote ya está donde Lomuto lo espera. Es la más simple pero la más vulnerable: en arreglos ya ordenados el pivote siempre cae en un extremo, lo que degenera el rendimiento a $O(n^2)$.

3.2.3.4. Variante de primer elemento

La variante de primer elemento simplemente intercambia el primer elemento con el último antes de partir. Resuelve el caso del arreglo invertido pero sigue siendo vulnerable al arreglo ya ordenado, que es el peor caso contrario.

3.2.3.5. Variante de elemento aleatorio

La variante de elemento aleatorio elige un índice al azar dentro del rango y lo mueve al final. Al no depender de la disposición actual del arreglo, rompe los patrones adversos que

afectan a los pivotes fijos y garantiza un buen rendimiento promedio sin importar el orden inicial de los datos.

3.2.3.6. Variante de mediana de tres

La variante de mediana de tres examina el primer, el del medio y el último elemento, los ordena entre sí mediante comparaciones directas, y usa la mediana como pivote moviéndola al final. Es la variante más robusta: evita los peores casos de los pivotes fijos, produce particiones más balanceadas que el pivote aleatorio en la mayoría de los casos, y no depende de un generador de números aleatorios.

3.2.3.7. Estructura recursiva

Cada función sigue la misma estructura: si $low < high$, se posiciona el pivote, se particiona el subarreglo obteniendo el índice final del pivote pi , y se llama recursivamente sobre los intervalos $[low, pi - 1]$ y $[pi + 1, high]$. El caso base es implícito: cuando $low \geq high$, el subarreglo tiene un solo elemento o está vacío, por lo que ya está ordenado y no hay nada que hacer.

La complejidad en el caso promedio es:

$$O(n \log n)$$

El peor caso es:

$$O(n^2)$$

y ocurre cuando el pivote siempre queda en un extremo, situación que las variantes de pivote aleatorio y mediana de tres mitigan considerablemente.

3.3. Algoritmos de Búsqueda y Selección

3.3.1. Búsqueda Binaria Recursiva

La búsqueda binaria recursiva se implementa mediante la función `recursive_binary_search`. La función recibe cinco parámetros:

- **deportistas:** Puntero al arreglo de estructuras deportista, que debe estar ordenado por ID.

- **left:** Índice del límite izquierdo del rango actual de búsqueda.
- **right:** Índice del límite derecho del rango actual de búsqueda.
- **targetId:** El ID del deportista que se desea encontrar.

La implementación se divide en tres partes principales, siguiendo la estrategia de divide y vencerás:

3.3.1.1. Caso base

El primer condicional evalúa si `left > right`. Esta condición indica que el rango de búsqueda se ha agotado sin encontrar el elemento, por lo que la función retorna `-1` para señalar que el ID no existe en el arreglo.

3.3.1.2. División

Si el rango aún contiene elementos (`left ≤ right`), el algoritmo calcula el punto medio del rango actual:

$$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$$

Esta expresión evita desbordamientos que podrían ocurrir con la forma clásica `(left + right) / 2` en arreglos muy grandes. El índice `mid` divide el problema en dos subproblemas de aproximadamente igual tamaño.

3.3.1.3. Conquista

Una vez dividido el problema, se compara el elemento en la posición `mid` con el `targetId`:

- **Elemento encontrado:** Si `deportistas[mid]->id == targetId`, se retorna el índice `mid` inmediatamente.
- **Buscar en la mitad izquierda:** Si `deportistas[mid]->id > targetId`, el elemento buscado debe estar en la mitad izquierda, ya que el arreglo está ordenado. Se realiza una llamada recursiva:

```
recursive_binary_search(deportistas, left, mid - 1, criteria, targetId)
```

- **Buscar en la mitad derecha:** Si `deportistas[mid]->id <targetId`, el elemento está en la mitad derecha. Se realiza la llamada recursiva:

```
recursive_binary_search(deportistas, mid + 1, right, criteria, targetId)
```

3.3.2. Búsqueda exponencial

La búsqueda exponencial implementa la estrategia divide y vencerás combinando dos fases: expansión exponencial para localizar un rango de búsqueda y luego búsqueda binaria recursiva dentro de ese rango.

3.3.2.1. Fase de expansión exponencial

El algoritmo comienza verificando si el elemento objetivo se encuentra en la primera posición. Si es así, retorna el índice 0 inmediatamente, logrando el mejor caso $O(1)$.

En caso contrario, inicia la fase de expansión con un límite `bound` establecido en 1. En cada iteración, duplica el valor del límite:

```
bound = bound * 2
```

Durante cada iteración, verifica si el elemento en la posición `bound` es mayor u igual al objetivo. Si lo es, ha encontrado un rango que contiene potencialmente al elemento buscado. El rango identificado se extiende desde `bound/2` hasta `bound` (o hasta `length - 1` si `bound` excede la longitud del arreglo).

La cantidad de iteraciones en esta fase es proporcional a $\log i$, donde i es la posición del elemento buscado, ya que cada duplicación del límite acota exponencialmente el espacio de búsqueda.

3.3.2.2. Fase de búsqueda binaria recursiva

Una vez identificado el rango, el algoritmo aplica búsqueda binaria de forma **completamente recursiva** sobre el intervalo `[bound/2, bound]`. La implementación utiliza una función auxiliar `exponencial_binary_search_recursive` que implementa divide y vencerás como se describió anteriormente.

3.3.3. Búsqueda por Interpolación

La búsqueda por interpolación se implementó como una alternativa optimizada a la búsqueda binaria para arreglos numéricos ordenados, utilizando una estrategia de divide y ven-

cerás mediante **división inteligente**.

A diferencia de la búsqueda binaria, que siempre divide el rango por la mitad, este algoritmo estima una posición probable del elemento objetivo basándose en los valores reales en los extremos del rango, permitiendo una división más cercana al objetivo.

3.3.3.1. Caso base

Si $low > high$, el rango se ha agotado sin encontrar el elemento, por lo que retorna -1 .

3.3.3.2. División

En lugar de calcular $mid = (left + right) / 2$ como en búsqueda binaria, la interpolación estima la posición usando la fórmula:

$$pos = low + \left[\frac{high - low}{highValue - lowValue} \cdot (targetValue - lowValue) \right] \quad (3.1)$$

Donde:

- **low / high:** Son los índices de los límites del rango actual.
- **lowValue / highValue:** Son los valores en dichos índices según el criterio de búsqueda.
- **targetValue:** Es el valor que se busca.

Esta fórmula mapea linealmente el valor buscado dentro del rango conocido de valores, produciendo una estimación más cercana al objetivo en datos con distribución uniforme. Esta es la división del problema: en lugar de un corte aritmético fijo, es un corte proporcional al valor.

3.3.3.3. Conquista

Una vez estimada la posición, se compara el valor encontrado con el objetivo:

- **Elemento encontrado:** Si los valores coinciden, retorna el índice.
- **Buscar en la mitad derecha:** Si el valor en pos es menor que el objetivo, se recursiona:

```
interpolation_search_recursive(deportistas, pos + 1, high, criteria, targetValue)
```

- **Buscar en la mitad izquierda:** Si el valor en `pos` es mayor que el objetivo, se recursiona:

```
interpolation_search_recursive(deportistas, low, pos - 1, criteria, targetValue)
```

3.3.4. Búsqueda por Rango

La búsqueda por rango implementa divide y vencerás para encontrar **todos** los elementos que coinciden con un valor específico, retornando los índices del primer y último elemento que poseen ese valor. Esto es especialmente útil cuando múltiples deportistas comparten el mismo puntaje o criterio de búsqueda.

3.3.4.1. Estructura recursiva

El algoritmo utiliza tres funciones recursivas auxiliares que implementan divide y vencerás en cada fase:

1. Fase de localización inicial (`find_any_recursive`):

Realiza una búsqueda binaria recursiva estándar para encontrar **cualquier** elemento que coincida con el valor objetivo. Una vez que encuentra un elemento coincidente, retorna su índice:

- **Caso base:** Si `low > high`, no se encontró nada, retorna `-1`.
- **División:** Calcula `mid = low + (high - low) / 2`.
- **Conquista:**
 - Si `deportistas[mid]` coincide con el objetivo, retorna `mid`.
 - Si el valor en `mid` es menor, recursa en la mitad derecha.
 - Si es mayor, recursa en la mitad izquierda.

2. Fase de expansión izquierda (`find_leftmost_recursive`):

Una vez localizado cualquier elemento coincidente, esta función busca recursivamente el **primer** elemento con ese valor expandiendo hacia la izquierda:

- **Caso base:** Si `low > high`, retorna `-1`.
- **División:** Calcula `mid = low + (high - low) / 2`.
- **Conquista:**

- Si `deportistas[mid]` coincide, continúa buscando en la mitad izquierda pero **recuerda este índice** como el candidato más a la izquierda.
- Si encuentra un elemento más a la izquierda, lo retorna; de lo contrario, retorna el índice actual.
- Si el valor es menor, busca a la derecha.
- Si es mayor, busca a la izquierda.

3. Fase de expansión derecha (`find_rightmost_recursive`):

Busca recursivamente el **último** elemento con el valor objetivo expandiendo hacia la derecha:

- **Caso base:** Si `low > high`, retorna `-1`.
- **División:** Calcula `mid = low + (high - low) / 2`.
- **Conquista:**
 - Si `deportistas[mid]` coincide, continúa buscando en la mitad derecha pero **recuerda este índice** como el candidato más a la derecha.
 - Si encuentra un elemento más a la derecha, lo retorna; de lo contrario, retorna el índice actual.
 - Si el valor es menor, busca a la derecha.
 - Si es mayor, busca a la izquierda.

3.3.4.2. Tipo de dato retornado

La función retorna una estructura **Range** que contiene dos campos:

- **start:** Índice del primer elemento que coincide con el valor objetivo.
- **end:** Índice del último elemento que coincide con el valor objetivo.

Si no se encuentra ningún elemento, ambos campos se establecen en `-1`.

3.3.5. Quick Select

Para obtener el elemento ubicado en una posición específica sin ordenar completamente el arreglo, se implementó el algoritmo *Quick Select* como una versión recursiva de *divide y vencerás*. Este algoritmo reutiliza la lógica de partición de Quick Sort, pero únicamente continúa la búsqueda sobre la partición que contiene el elemento deseado, reduciendo significativamente el trabajo realizado.

3.3.5.1. Selección de pivote con mediana de tres

Antes de particionar, el algoritmo utiliza la estrategia de **mediana de tres** para elegir el pivote:

- Se examinan tres elementos: el primero (**left**), el del medio, y el último (**right**).
- Se ordenan estos tres elementos entre sí mediante comparaciones directas.
- La mediana de estos tres se coloca en la última posición (**right**), donde la partición la espera.

Esta estrategia evita los peores casos de pivotes fijos (primer o último elemento) y produce particiones más balanceadas que un pivote aleatorio en la mayoría de los casos prácticos.

3.3.5.2. Partición de Lomuto

La operación principal es la función **partition**, basada en el esquema de Lomuto. Durante el recorrido del subarreglo:

- Se mantiene un índice i que marca el límite entre elementos que deben ir a la izquierda del pivote y los que no.
- Se comparan todos los elementos (excepto el pivote) con el pivote, decidiendo si preceden o no según el criterio y orden seleccionados.
- Los elementos que deben estar a la izquierda se intercambian hacia esa posición.
- Al final, el pivote se coloca en su posición definitiva.

La comparación se realiza mediante **compare_by_criteria**, permitiendo trabajar con múltiples criterios (ID, puntaje, competencias), y el orden (ascendente o descendente) se controla mediante el parámetro **order**.

3.3.5.3. Estructura recursiva de divide y vencerás

La función **quick_select_recursive** implementa el ciclo de divide y vencerás:

- **Caso base:** Si el subarreglo contiene un único elemento (**left == right**), se retorna directamente sin más procesamiento.
- **División:** Se aplica mediana de tres y se particiona el subarreglo, obteniendo el índice del pivote (**pivotIndex**).

▪ **Conquista:** Se evalúan tres casos:

- Si $k = \text{pivotIndex}$, el elemento buscado fue encontrado en su posición correcta.
- Si $k < \text{pivotIndex}$, el elemento está en la mitad izquierda, se recursa:

textttquick_select_recursive(arr, left, pivotIndex - 1, k, ...)

- Si $k > \text{pivotIndex}$, el elemento está en la mitad derecha, se recursa:

textttquick_select_recursive(arr, pivotIndex + 1, right, k, ...)

3.3.5.4. Complejidad

La complejidad temporal promedio es:

$$O(n)$$

ya que en cada iteración solo se continúa explorando una partición del arreglo, y el tamaño del subarreglo se reduce aproximadamente por un factor constante en promedio.

Sin embargo, el peor caso ocurre cuando el pivote siempre queda en un extremo, resultando en:

$$O(n^2)$$

aunque este comportamiento es poco frecuente en datos aleatorios y mitiga considerablemente el uso de mediana de tres.

La complejidad espacial es $O(\log n)$ por las llamadas recursivas apiladas en el stack.

3.4. Aplicación práctica

3.4.1. Ranking de los mejores N deportistas

Para obtener el ranking de los N mejores deportistas según puntaje, se implementó la función `get_top_n_athletes`, que recibe el arreglo de deportistas, su cantidad total y el valor N . La función reserva un arreglo de N posiciones y lo completa llamando a `get_kth_athlete` con $k = 1, 2, \dots, N$, obteniendo cada puesto de forma independiente mediante Quick Select.

A diferencia de ordenar el arreglo completo, Quick Select localiza el k -ésimo elemento sin necesidad de ordenar todos los datos. Su costo promedio es:

$$O(n)$$

por llamada, lo que resulta en un costo total de:

$$O(N \cdot n)$$

para el ranking completo, frente al costo:

$$O(n \log n)$$

de un ordenamiento completo. Esto lo hace más eficiente cuando N es pequeño respecto al total de deportistas.

La función `show_ranking` se encarga de cargar los datos desde el archivo CSV, invocar `get_top_n_athletes` para obtener los resultados y mostrarlos utilizando las funciones de formato ya existentes en el sistema.